

第3章 单片机的指令系统及汇编语言程序设计

第2章介绍了AT89系列单片机的硬件结构，但是，光有硬件单片机是不可能工作的，还需要有相应的软件配合。单片机的软件就是利用其指令系统所编写的程序。可以利用单片机的指令直接编写程序。用这种方式编写的程序称为手编程序。这种用机器语言编写的程序，单片机可以直接执行。但直接用机器语言编写程序是一件很烦琐的工作，需要耗费大量的人力和时间，而且又容易出错，纠错也非常困难。

为了编写程序方便和提高效率，人们用一些约定的文字、符号和数字按规定的格式来表示各种不同的指令，然后再用这些约定符号表示的指令来编写程序，这就是汇编语言。使用汇编语言编写的程序称为汇编程序。汇编程序编写比直接使用机器语言方便得多，但是汇编语言程序需要进行翻译（也就是汇编），单片机才能执行。

因为AT89系列单片机指令系统与51系列单片机指令系统完全兼容，所以本章以51系列单片机为例介绍其指令系统和汇编语言程序设计方法。

3.1 51 系列单片机指令系统概述

指令就是要计算机执行某种操作的命令，每一条指令可完成一个独立的算术或逻辑运算。一台计算机中所有指令的集合，称为这台计算机的指令系统。指令通常由操作码字段和操作数地址码字段组成。操作码字段表征指令的操作特性与功能，而地址码字段通常指定参与操作的操作数的地址。一条指令的结构用如下形式表示：

操作码字段 OP 地址码字段 A

在51单片机中的指令系统中，有单字节、双字节、三字节等不同长度的指令。

单字节指令只有1字节，操作码和操作数都在这个字节中。在单字节指令中，一部分指令的操作数是默认的，不需要在指令中指出；另一部分指令的操作数在寄存器中。因为51单片机的寄存器组有8个寄存器，所以只需要3位编码。这些操作数编码可以和操作码一起存放在1字节中。51单片机的指令系统共有单字节指令49条。

双字节指令包括2字节，其中第一个字节是操作码，第二个字节是操作数。例如：立即数加法指令 `ADD A, #data`。其中 `data` 表示一个8位的立即数，需要1字节。51指令系统共有双字节指令45条。

三字节指令中，操作码占1字节，操作占2字节，其中操作既可能是数据，也可能是地址。例如：逻辑或操作指令 `ORL direct, #data`，直接寻址单元与立即数进行与操作。其中 `direct` 是一个直接地址，需要1字节，`data` 是一个立即数，需要1字节，加上操作码共需3字节。51指令系统共有三字节指令45条。

3.2 51 系列单片机的寻址方式

几乎所有的指令都需要操作数，如何得到操作数的地址，是指令执行中一个关键的问题。

题。我们称指令中的操作数字段为操作数的形式地址，操作数实际所在的区域称之为操作数的实际地址。所谓寻址，就是根据操作数的形式地址得到操作数的实际地址的过程。通过不同方法得到实际地址，就有了不同的寻址方式。51 系列单片机共有以下 7 种寻址方式。

3.2.1 立即寻址

立即寻址就是操作数在指令中直接给出，也就是指令的地址即操作数的地址，因此把这种寻址方式称之为立即寻址，该操作数称之为立即数。为了表示指令中的立即数，在操作数前面加“#”标志。

```
MOV A, #90H
```

这是一条立即寻址方式的指令，指令功能是把 90H 这个数据送到累加寄存器 A。

3.2.2 寄存器寻址

寄存器寻址就是操作数在寄存器中，因此形式地址就是一个寄存器名，指定了寄存器就能得到操作数。例如指令：

```
MOV A, R0
```

其功能是把寄存器 R0 的内容传送到累加寄存器 A 中。由于操作数在 R0 中，因此在指令中指定了 R0，就能从中取得操作数，所以是寄存器寻址方式。

寄存器寻址方式的寻址范围包括：

1) 通用寄存器。共有 4 组 32 个通用寄存器，但在任何时刻只使用当前组寄存器组。所以在指令中这些寄存器的名字只有 8 个，即 R0 ~ R7。PSW 寄存器中 RS1、RS0 组合的值 (0 ~ 3) 决定了使用 4 组寄存器中的哪一组寄存器，如果要更换寄存器组，就应该重新设置 RS1、RS0 的值。

2) 部分专用寄存器。例如累加器 A 和 B 寄存器，以及数据指针 DPTR 寄存器等。

3.2.3 直接寻址

直接寻址的操作数在内存单元中，在指令中操作数直接以内存单元地址的形式给出。例如指令：

```
ADD A, 56H
```

其功能是把内部 RAM 56H 单元中的数据与累加寄存器 A 相加之和送到累加寄存器 A 中。

直接寻址方式的寻址范围只限于内部 RAM，包括：

1) 低 128 单元。在指令中直接以单元地址形式给出。

2) 专用寄存器。专用寄存器除了以单元地址形式给出外，还可以以寄存器符号形式给出。直接寻址是访问专用寄存器的唯一方法。例如指令：

```
MOV A, SBUF
```

其中，SBUF 是专用寄存器名。

3.2.4 寄存器间接寻址

寄存器中存放的是操作数的地址，我们称之为寄存器间接寻址，即操作数是通过寄存器

间接得到的。

在 51 单片机中，能够用来间接寻址的寄存器有：用户所选定的工作寄存器组的 R0、R1，16 位的数据指针 DPTR。寄存器间接寻址方式的寻址范围包括：

1) 内部数据 RAM 的寄存器间接寻址。采用寄存器 R0、R1，对低 128 个单元进行寻址。例如：

```
MOV A, @R0
```

其功能是将 R0 的内容为地址的内部 RAM 对应单元内容送到累加寄存器 A 中。

2) 外部数据 RAM 的寄存器间接寻址。有两种形式：一种是采用 R0、R1 作为间址寄存器。这时 R0 或 R1 提供低 8 位地址，访问外部 RAM 的低 256 字节；另一种是采用 16 位的 DPTR 作为间址寄存器，可以访问整个外部 64KB RAM。例如：

```
MOVX A, @R0
```

```
MOVX A, @DPTR
```

3.2.5 相对寻址

相对寻址是把指令中给定的地址偏移量与 PC 的当前值（读出当前双字节或三字节的跳转指令后，PC 已经指向了下条指令）相加，得到真正的程序转移地址。相对寻址方式主要是解决程序转移而专门设置的。例如：

```
JC 80H
```

若 C = 0，则 PC 值不变；若 C = 1，则以当前 PC 值为基地址，加上偏移量 80H 得到新的 PC 值。这里的偏移量是一个带符号的二进制补码数，所能表示的数的范围是 -128 ~ +127，因此相对转移是以转移指令的下条指令为基点，向前最大可能转移 127 字节，向后最大可能转移 128 字节。

3.2.6 变址寻址

变址寻址方式是以 DPTR 或 PC 作为基址寄存器，以累加器 A 作为偏移量寄存器，将一个基址寄存器的内容与偏移量寄存器的内容之和作为操作数地址。变址寻址是为了访问程序存储器中的数据表格。例如：

```
MOVC A, @A + DPTR
```

设累加寄存器 A 为 10H，寄存器 DPTR 为 1000H，程序存储器中 1010H 单元内容为 45H，则上面程序语句的功能是将 A 的内容与 DPTR 的内容相加形成操作数地址 1010H，把该地址中的数据传送到累加器 A，即 $A \leftarrow (A) + (DPTR)$ 。结果，累加寄存器 A 为 45H。

51 指令系统的变址寻址方式有如下特点。

1) 变址寻址方式只能对程序存储器进行寻址，寻址范围达 64KB。

2) 变址寻址的指令只有 3 条：

```
MOVC A, @A + DPTR
```

```
MOVC A, @A + PC
```

```
JMP @A + DPTR
```

尽管变址寻址方式较为复杂，但变址寻址的指令却都是单字节指令。

3.2.7 位寻址

51 指令系统有位处理功能,可以对数据位进行操作,因此就有相应的位寻址方式。位寻址的范围包括:

- 1) 内部 RAM 中的位寻址区。单元地址为 20H ~ 2FH,共 16 个单元 128 位,位地址是 00H ~ 7FH。
- 2) 专用寄存器的可寻址位。可以位寻址的专用寄存器共有 11 个,实有寻址位 83 位。

3.3 51 系列单片机指令系统

51 单片机指令系统共有 111 条指令,按功能分类,可分为 5 大类:

数据传送类指令 (29 条)、算术操作类指令 (24 条)、逻辑运算类指令 (25 条)、控制转移类指令 (17 条) 和位操作类指令 (17 条)。

在学习指令之前,对指令中使用的符号做简单的说明。

Rn: 当前寄存器区的 8 个工作寄存器 R0 ~ R7 ($n=0 \sim 7$)。

Ri: 当前选中的寄存器区中可作为间接寻址寄存器的 2 个寄存器 R0、R1 ($i=0, 1$)。

Direct: 直接地址,即 8 位的内部数据存储单元或特殊功能寄存器的地址。

#data: 包含在指令中的 8 位立即数。

#data16: 包含在指令中的 16 位立即数。

rel: 相对转移指令中的偏移量,为 8 位的带符号补码数。

DPTR: 数据指针寄存器,可用作 16 位的数据地址寄存器。

bit: 内部 RAM 或特殊功能寄存器中的直接寻址位。

A: 累加寄存器。

ACC: 直接寻址方式的累加器。

B: 寄存器 B。

C (或 CY): 进位标志位或位处理机中的累加器。

addr11: 11 位目的地址。

addr16: 16 位目的地址。

@: 间接寻址寄存器前缀,如 @ Ri、@ A + DPTR。

/: 加在位地址的前面,表示对该位取反。

(X): 名字为 X 的寄存器或某存储单元内容。

((X)): 表示某个存储单元的内容,其地址存放在名字为 X 的寄存器或某存储单元。

← 箭头左边的内容被箭头右边的内容所取代。

→ 箭头右边的内容被箭头左边的内容所取代。

3.3.1 数据传送指令

数据传送指令属于复制,传送后,源操作数不变,不影响别的寄存器和标志。传送指令包括 “MOV”、“MOVB”、“MOVW”。

数据传送指令的约定是从右向左,左边是目的操作数,右边是源操作数。源操作数可以

是累加寄存器、通用寄存器、直接地址、间接地址和立即数。目的操作数除了不能用立即数，其他和源操作数一样。

1. 片内数据存储器传送指令

片内数据传送指令 MOV 的功能是单片机内部数据存储器内的数据相互传送。其形式为：
MOV <目的操作数> , <源操作数>

这里的目的操作数可以是累加寄存器、通用寄存器、直接地址、间接地址；源操作数可以是累加寄存器、通用寄存器、直接地址、间接地址、立即数。指令格式如下：

(1) 以 A 为目的操作数

MOV A, Rn ; $A \leftarrow Rn$
MOV A, direct ; $A \leftarrow (direct)$
MOV A, @Ri ; $A \leftarrow ((Ri))$
MOV A, #data ; $A \leftarrow data$

Rn 和 Rn 、 Rn 和 $@Ri$ 不能相互配对。

即一条指令中不能同时出现工作寄存器。

(2) 以 Rn 为目的操作数

MOV Rn, A ; $Rn \leftarrow A$
MOV Rn, direct ; $Rn \leftarrow direct$
MOV Rn, #data ; $Rn \leftarrow data$

(3) 以直接地址 direct 为目的操作数

MOV direct, A ; $direct \leftarrow A$
MOV direct, Rn ; $direct \leftarrow Rn$
MOV direct2, direct1 ; $direct2 \leftarrow (direct1)$
MOV direct, @Ri ; $direct \leftarrow ((Ri))$
MOV direct, #data ; $direct \leftarrow data \#data$

(4) 以间接地址 @ Ri 为目的操作数

MOV @Ri, A ; $(Ri) \leftarrow A$
MOV @Ri, direct ; $(Ri) \leftarrow (direct)$
MOV @Ri, #data ; $(Ri) \leftarrow data \#data$

(5) 以 DPTR 为目的操作数

MOV DPTR, #data16 ; $DPTR \leftarrow data16 \#data16$

2. 外部数据存储器传送指令

外部数据传送指令 MOVX 的功能是单片机外部数据存储器的数据和 A 寄存器相互传送。目的操作数可以是 A 寄存器或寄存器间接寻址，源操作数也可以是 A 寄存器或寄存器间接寻址。它的指令格式如下：

MOVX A, @DPTR ; $A \leftarrow ((DPTR))$
MOVX @DPTR, A ; $(DPTR) \leftarrow (A)$
MOVX A, @Ri ; $A \leftarrow ((Ri))$
MOVX @Ri, A ; $(Ri) \leftarrow (A)$

3. 程序存储器数据传送指令

程序存储器数据传送指令 MOVC 功能是将程序存储器的存储单元的内容传送到寄存器 A 中，指令格式如下：

MOVC A, @A + DPTR ; $A \leftarrow ((A) + (DPTR))$
MOVC A, @A + PC ; $A \leftarrow ((A) + (PC))$

例1. 1> 将R0内容送至Rb.

```
MOV A, R0.
```

```
MOV Rb., A.
```

2> 将片内RAM 20H单元内容送至片外60H单元.

```
MOV A, 20H
```

```
MOV R0, #60H
```

```
MOV @R0, A.
```

3> 将片外RAM 1000H单元内容送至片内20H单元.

```
MOV DPTR, #1000H.
```

```
MOV A, @DPTR
```

```
MOV 20H, A.
```

4> 将ROM 2000H单元内容送至片内RAM 30H单元.

```
MOV A, #0
```

```
MOV DPTR, #2000H
```

```
MOVC A, @A+DPTR
```

```
MOV 30H, A.
```

例 下面程序能根据累加器A的值0, 2, 4, 6 转移到相应的TAB0~TAB6分支执行.

```
MOV DPTR, #TABLE ; 表首地址送DPTR.
```

```
JMP @A+DPTR ; 根据A值转.
```

```
TAB0: AJMP TAB0 ; 当(A)=0时转TAB0.
```

```
AJMP TAB2 ; 当(A)=2时转TAB2
```

```
AJMP TAB4 ; 当(A)=4 ---
```

```
AJMP TAB6 ; ---
```

【例 3-1】 设一个以字节为单元的表放在程序存储器内，其起始地址在 DPTR 寄存器中，A 寄存器内容为 15，则执行指令

MOVC A, @ A + DPTR

后，该表格的 15 单元的内容将放入寄存器 A 中。

4. 堆栈指令

(1) 进栈指令

进栈指令 PUSH 的功能是将一个数据压入栈中。先将栈指针 SP 加 1，然后把该数据送到栈指针 SP 指示的内部 RAM 单元中。进栈指令经常用于子程序调用时保护寄存器的内容，防止这些寄存器的内容在子程序中被破坏。指令格式如下：

PUSH direct ; SP ← (SP) + 1, (SP) ← (direct)

(2) 出栈指令

出栈指令 POP 的功能是将一个数据从栈中弹出。首先将 SP 指示的内部 RAM 单元送到目标寄存器或存储单元中，然后将指针 SP 减 1。出栈指令经常用于子程序运行结束时恢复所保护寄存器的内容，这些寄存器可以在主程序中完整地继续使用。指令格式如下：

POP direct ; (direct) ← (SP), SP ← (SP) - 1

在子程序中保护寄存器和恢复寄存器的 PUSH 和 POP 语句必须反序排列，也就是按照后进先出的原则。

5. 数据交换指令

(1) 字节交换指令

字节交换指令 XCH 是将源操作数的内容送到目的操作数中，将目的操作数送到源操作数中。源操作数只能是 A 寄存器，目的操作数可以是通用寄存器、直接地址、间接地址。指令格式如下：

XCH A, Rn ; (A) ↔ (Rn)
XCH A, direct ; (A) ↔ (direct)
XCH A, @ Ri ; (A) ↔ (Ri)

【例 3-2】 设 (R1) = 30H, (30H) = 45H, (A) = 7FH，则执行指令

XCH A, @ R1

后，(A) = 45H，而 (30H) = 7FH，从而实现了累加器 A 与内部数据存储器 RAM 中 30H 单元的数据交换。

(2) 半字节交换指令

半字节交换指令 XCHD 是将源操作数的低 4 位（半字节）送到目的操作数的低 4 位中，目的操作数低 4 位（半字节）送到源操作数的低 4 位中。源操作数只能是 A 寄存器，目的操作数可以是间接地址。指令格式如下：

XCHD A, @ Ri ; (A) 3~0 ↔ ((Ri)) 3~0

【例 3-3】 设 (30H) = 6FH, (R0) = 30H, (A) = 0F6H，则执行指令：

XCHD A, @ R0

结果：(A) = 0FFH, (30H) = 66H。

(3) 累加器 A 中高 4 位与低 4 位交换指令

该指令所执行的操作是累加器 A 中的高 4 位与低 4 位的内容互换，其结果仍存放在累加器 A 中，结果不影响标志寄存器 PSW。指令格式如下：

SWAP A ; $(A)_{7-4} \leftrightarrow (A)_{3-0}$

【例 3-4】 设 $(A) = 0A5H$ (10100101B)，执行指令：

SWAP A

结果： $(A) = 5AH$ (01011010B)。

3.3.2 算术运算指令

1. 加法指令

加法指令 ADD 所完成的功能是把源操作数与累加器 A 的内容相加，将结果保存在累加器 A 中。指令格式如下：

```
ADD A, #data ; A ← (A) + data
ADD A, direct ; A ← (A) + (direct)
ADD A, @ Ri ; A ← (A) + ((Ri))
ADD A, Rn ; A ← (A) + (Rn)
```

其结果影响标志寄存器中的进位 CY、辅助进位 AC、溢出 OV、奇偶 P 标志。

如果在运算过程中，位 7 产生进位，则置“1”进位标志 CY；否则，清“0”CY。如果在运算过程中，低 4 位向高 4 位产生进位，置“1”辅助进位标志 AC；否则，清“0”AC。如果在运算过程中，位 6 有进位，而位 7 没有进位，或者位 7 有进位，而位 6 没有，则溢出标志位 OV 置“1”；否则，清“0”OV。溢出标志位 OV 的状态，只有在带符号数加法运算时才有意义。当两个带符号数相加时， $OV = 1$ ，表示加法运算超出了累加器 A 所能表示的带符号数的有效范围（-128 ~ 127）。运算结果如果有偶数个 1，则置“1”奇偶 P 标志；否则，清“0”P。

【例 3-5】 有程序段

```
MOV A, #0A9H
ADD A, #0B8H
```

执行完后， $(A) = 61H$ ，进位标志 CY 为 1，辅助标志 AC 为 1（9+8 结果超过 16），溢出标志 OV 为 1（作为 8 位有符号数，A9H 和 B9H 都是负数，相加结果小于 -128），奇偶标志 P 为 1（A 中有 3 个 1）。

2. 带进位加法指令

带进位加法指令 ADDC 与前述加法指令的区别仅为考虑进位位，其他与加法指令相同。所完成的功能是把源操作数与累加器 A 的内容相加后再加上进位标志 CY，将结果保存在累加器 A 中。其结果影响标志寄存器中的进位、辅助进位、溢出、奇偶标志。指令格式如下：

```
ADDC A, #data ; A ← (A) + data + (CY)
ADDC A, direct ; A ← (A) + (direct) + (CY)
ADDC A, @ Ri ; A ← (A) + ((Ri)) + (CY)
ADDC A, Rn ; A ← (A) + (Rn) + (CY)
```

【例 3-6】 利用 ADDC 指令可以进行多字节的加法运算。

设有两个 16 位数相加，被加数的高 8 位放在 41H，低 8 位放在 40H，加数的高 8 位放在 43H，低 8 位放在 42H，和的低 8 位存放在 50H，高 8 位存放在 51H，进位位存放在 52H。可编程序如下：

```
MOV A, 40H ; A ← 被加数低 8 位
```

ADD A, 42H	; 与加数低 8 位相加
MOV 50H, A	; 和的低 8 位存入 50H
MOV A, 41H	; A←被加数高 8 位
ADDC A, 43H	; 被加数高 8 位与加数高 8 位以及低位来的进位相加
MOV 51H, A	; 和的高 8 位存入 51H 单元
MOV A, #00H	; A←00H
ADDC A, #00H	; A←(A) + 00H + 高 8 位来的进位
MOV 52H, A	; 进位位 CY 内容存入 52H 单元

3. 加一指令

加一指令 INC 的功能是将操作数加 1 再送回操作数。操作数可以是直接地址、间接地址、通用寄存器。**加一指令不影响标志位**。指令格式如下：

INC A	; A←(A) + 1
INC direct	; direct←(direct) + 1
INC @ Ri	; (Ri) ← ((Ri)) + 1
INC Rn	; Rn←(Rn) + 1
INC DPTR	; DPTR←(DPTR) + 1

4. 带借位减法指令

带借位减法指令 SUBB 所完成的功能是累加器 A 减去源操作数（立即数、直接地址、间接地址、通用寄存器）再减去进位标志 CY，将结果保存在累加器 A 中。其结果影响标志寄存器中的进位 CY、辅助进位 AC、溢出 OV、奇偶 P 标志。

如果在运算过程中，位 7 产生借位，则置“1”进位标志 CY；否则，清“0”CY。在减法运算中，CY 成了借位标志。

如果在运算过程中，低 4 位向高 4 位产生借位，置“1”辅助进位标志 AC；否则，清“0”AC。在减法运算中，AC 成了辅助借位标志。

溢出标志位 OV 的状态，只有在带符号数减法运算时才有意义。当两个带符号数相减时，OV = 1，表示加法运算超出了累加器 A 所能表示的带符号数的有效范围。指令格式如下：

SUBB A, #data	; A←(A) - data - (CY)
SUBB A, direct	; A←(A) - (direct) - (CY)
SUBB A, @ Ri	; A←(A) - ((Ri)) - (CY)
SUBB A, Rn	; A←(A) - (Rn) - (CY)

【例 3-7】 设 (45H) = 0AAH, (47H) = 66H, 试编写 45H 内容减去 47H 内容后，结果再存入 49H 单元的程序。

```
MOV 45H, #0AAH
MOV 47H, #66H
MOV A, 45H ; 操作数对齐, ; 向 A 设置被减数
CLR C      ; 清除进位标志 CY
SUBB A, 47H ; A←(A) - (47H) - (CY)
MOV 49H, A  ; 将结果置于地址为 49H 的内部 RAM 存储单元中
```

执行以上程序后，(49H) = 44H, CY = 0, OV = 1, AC = 0。

5. 减一指令

减一指令 DEC 的功能是将操作数减 1 再送回操作数。操作数可以是直接地址、间接地址、通用寄存器。减一指令不影响标志位。指令格式如下：

```
DEC A           ; A ← (A) - 1
DEC direct      ; direct ← (direct) - 1
DEC @ Ri        ; (Ri) ← ((Ri)) - 1
DEC Rn          ; Rn ← (Rn) - 1
DEC DPTR       ; DPTR ← (DPTR) - 1
```

6. 十进制调整指令

十进制调整指令用于对 BCD 码十进制数加法运算结果的内容修正。指令格式如下：

DA A

虽然我们用 4 位二进制来表示十进制的一个数字，但是二进制数的加法运算原则并不能适用于十进制数的加法运算，有时会产生错误结果。例如， $6 + 7 = 13$ 也就是 BCD 码 6 加上 BCD 码 7 得到的结果应该是 3，并伴随着一个辅助进位。但在二进制中 6 加上 7 为 1101，不产生辅助进位。又如， $9 + 8 = 17$ 也就是 BCD 码 9 加上 BCD 码 8 得到的结果应该是 7，并伴随着一个辅助进位。但在二进制中 8 加上 9 为 0001，产生辅助进位。产生错误结果的根本原因是 BCD 码加法是满 10 就产生一个进位，而 4 位二进制数相加是满 16 才产生进位，也就是在 4 位二进制相加一旦产生进位，多减去了 6，必须进行加 6 修正，才能和 BCD 码加法相吻合，两个 BCD 码按二进制相加之后，经本指令的调整，才能得到正确的压缩 BCD 码的和。

调整的方法是把结果加 6 调整，即十进制调整修正，具体方法如下：

- 1) 累加器低 4 位大于 9 或辅助进位位 $AC = 1$ ，则进行低 4 位加 6 修正。
- 2) 累加器高 4 位大于 9 或进位位 $CY = 1$ ，则进行高 4 位加 6 修正。
- 3) 累加器高 4 位为 9，低 4 位大于 9，则高 4 位和低 4 位分别加 6 修正。

具体是通过执行指令：

DA A

来自动实现的。

【例 3-8】 $(A) = 56H$ ， $(R5) = 67H$ ，把它们看作两个压缩的 BCD 数，进行 BCD 数的加法。执行指令：

```
MOV A, #56H
MOV R5, #67H
ADD A, R5
DA A
```

由于高、低 4 位分别大于 9，所以要分别加 6 进行十进制调整对结果进行修正。结果为： $(A) = 23H$ ， $CY = 1$ 。可见， $56 + 67 = 123$ ，结果是正确的。

7. 乘法指令

乘法指令只有一种格式：

MUL AB ; $(A) \times (B) \rightarrow BA$

如果积大于 255，则置“1”溢出标志位 OV。

【例 3-9】

```
MOV A, #60H
```

```
MOV B, #60H
```

```
MUL AB
```

执行后的结果为 (A) = 00H, (B) = 24H, OV = 1。

8. 除法指令

除法指令只有一种格式：

```
DIV AB ; (A) ÷ (B) → A, 余数 → B
```

如果 B 的内容为“0”，则存放结果的 A、B 中的内容不定，并置“1”溢出标志位 OV。

【例 3-10】

```
MOV A, #67
```

```
MOV B, #20
```

```
DIV AB
```

执行后的结果为：(A) = 03H, (B) = 07H, OV = 0。

3.3.3 逻辑操作指令

1. 累加器 A 清 0 指令

累加器 A 清 0 指令 CLR 的功能是将累加器 A 的值清 0。指令格式如下：

```
CLR A ; A ← 0
```

2. 累加器 A 取反指令

累加器 A 取反指令 CPL 的功能是将累加器 A 的数据取反后再送回累加器 A 中。指令格式如下：

```
CPL A ; A ← ¬(A)
```

【例 3-11】

```
MOV A, #3EH
```

```
CPL A
```

执行后的结果为 (A) = 0C1H。

3. 累加器 A 循环左移指令

累加器 A 循环左移指令 RL 的功能是将累加器 A 的数据循环左移 1 位。最高位左移后将进入最低位。

该指令对 P 标志位有影响，如图 3-1 所示。

指令格式如下：

```
RL A ; An+1 ← An, A0 ← A7
```

【例 3-12】

```
MOV A, #3EH
```

```
RL A
```

执行后的结果为 (A) = 7CH。

4. 累加器 A 带进位位 CY 循环左移指令

累加器 A 带进位位 CY 循环左移指令 RLC 的功能是将累加器 A 中的内容与进位标志位 CY 一起循环左移 1 位。最高位左移后，进入 CY 标志位，CY 标志位进入最低位。该指令对 P 标志位有影响，如图 3-2 所示。

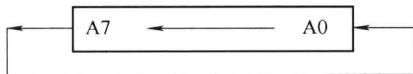


图 3-1 A 循环左移 1 位

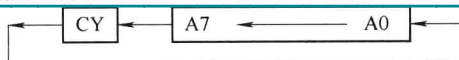


图 3-2 A 带进位循环左移 1 位

指令格式如下：

RLC A ; $A_{n+1} \leftarrow A_n$, $CY \leftarrow A_7$, $A_0 \leftarrow CY$

【例 3-13】

MOV A, #3EH

SETB C ; 将进位标志 CY 置 1

RLC A

执行后的结果为：(A) = 7DH, CY = 0, P = 0。

5. 累加器 A 循环右移指令

累加器 A 循环右移指令 RR 的功能是将累加器 A 的数据循环右移 1 位。最低位右移后将进入最高位。该指令对 P 标志位有影响，如图 3-3 所示。

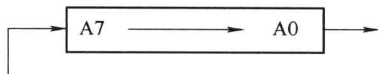


图 3-3 A 循环右移 1 位

指令格式如下：

RR A ; $A_n \leftarrow A_{n+1}$, $A_7 \leftarrow A_0$

【例 3-14】

MOV A, #8EH

RR A

执行后的结果为 (A) = 47H。

6. 累加器 A 带进位位 CY 循环右移指令

累加器 A 带进位位 CY 循环右移指令 RRC 的功能是将累加器 A 中的内容与进位标志位 CY 一起循环右移 1 位。最低位右移后，进入 CY 标志位，CY 标志位进入最高位。该指令对 P 标志位有影响，如图 3-4 所示。

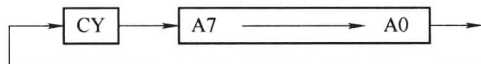


图 3-4 A 带进位循环右移 1 位

指令格式如下：

RRC A ; $A_n \leftarrow A_{n+1}$, $CY \leftarrow A_0$, $A_7 \leftarrow CY$

【例 3-15】

MOV A, #8EH

SETB C ; 将进位标志 CY 置 1

RRC A

执行后的结果为：(A) = C7H, CY = 0, P = 1。

7. 逻辑与指令

逻辑与指令 ANL 是双操作数指令，其目的操作数可以是累加寄存器 A 或直接地址，所完成的功能是把源操作数与目的操作数进行位与运算（第 0 ~ 7 位分别进行逻辑与运算），将结果保存在目的操作数中。

ANL A, #data	; $A \leftarrow (A) \wedge \text{data}$
ANL A, direct	; $A \leftarrow (A) \wedge (\text{direct})$
ANL A, @ Ri	; $A \leftarrow (A) \wedge ((Ri))$
ANL A, Rn	; $A \leftarrow (A) \wedge (Rn)$
ANL direct, A	; $\text{direct} \leftarrow (\text{direct}) \wedge (A)$
ANL direct, #data	; $\text{direct} \leftarrow (\text{direct}) \wedge \text{data}$

【例 3-16】

```
MOV A, #8EH
MOV R1, 70H
MOV @R1, #88H
ANL A, @R1
```

执行后的结果为: (A) = 88H, P = 0。

8. 逻辑或指令

逻辑或指令 ORL 是双操作数指令, 其目的操作数可以是累加寄存器 A 或直接地址, 所完成的功能是把源操作数与目的操作数进行位或运算 (第 0 ~ 7 位分别进行逻辑或运算), 将结果保存在目的操作数中。指令格式如下:

ORL A, #data	; $A \leftarrow (A) \vee \text{data}$
ORL A, direct	; $A \leftarrow (A) \vee (\text{direct})$
ORL A, @Ri	; $A \leftarrow (A) \vee ((Ri))$
ORL A, Rn	; $A \leftarrow (A) \vee (Rn)$
ORL direct, A	; $\text{direct} \leftarrow (\text{direct}) \vee (A)$
ORL direct, #data	; $\text{direct} \leftarrow (\text{direct}) \vee \text{data}$

【例 3-17】

```
MOV A, #8FH
MOV R1, 70H
MOV @R1, #0F8H
ORL A, @R1
```

执行后的结果为: (A) = 0FFH, P = 0。

9. 逻辑异或指令

逻辑异或指令 XRL 是双操作数指令, 其目的操作数可以是累加寄存器 A 或直接地址, 所完成的功能是把源操作数与目的操作数进行位异或运算 (第 0 ~ 7 位分别进行逻辑异或运算), 将结果保存在目的操作数中。指令格式如下:

XRL A, #data	; $A \leftarrow (A) \oplus \text{data}$
XRL A, direct	; $A \leftarrow (A) \oplus (\text{direct})$
XRL A, @Ri	; $A \leftarrow (A) \oplus ((Ri))$
XRL A, Rn	; $A \leftarrow (A) \oplus (Rn)$
XRL direct, A	; $\text{direct} \leftarrow (\text{direct}) \oplus (A)$
XRL direct, #data	; $\text{direct} \leftarrow (\text{direct}) \oplus \text{data}$

【例 3-18】

```
MOV A, #99H
MOV R1, 70H
MOV @R1, #0FFH
XRL A, @R1
```

执行后的结果为: (A) = 66H, P = 0。

3.3.4 控制转移指令

程序的顺序执行是由 PC 自动加 1 实现的。要改变程序的执行顺序, 实现分支转向, 应通过强迫改变 PC 寄存器内容的方法来实现, 这就是控制转移指令的实现基础。控制转移指

令分为：无条件转移指令、有条件转移指令和子程序调用及返回指令。

1. 无条件转移指令

(1) 长转移指令

长转移指令 **LJMP** 是单操作数指令，功能是把操作数所表示的 16 位地址送入 PC 寄存器，也就是要执行的下一条指令是操作数所指出地址的指令，从而实现程序的跳转。由于本指令的操作数是 16 位地址，所以可以实现在整个 64KB 程序空间的任意跳转，故称之为长转移指令。长转移指令需要 3 字节。指令格式如下：

LJMP addr16 ; PC ← addr16

【例 3-19】

```
ORG 0000H
LJMP START
ORG 0030H
START: LJMP LAB
ORG 7000H
LAB: MOV A, R4
```

在上面的例题中，用标号来表示指令的地址，START 标号表示的地址是 0030H，而 LAB 所表示的地址是 7000H。在程序中，一般都不在跳转指令中直接写地址值，而是采用标号来表示地址。

(2) 绝对转移指令

绝对转移指令 **AJMP** 也是单操作数指令，功能是把操作数所表示的 11 位地址送入 PC 寄存器的低 11 位，高 5 位不变，从而实现程序在 2KB 范围的跳转。绝对转移指令需要 2 字节。指令格式如下：

AJMP addr11 ; PC ← (PC) + 2, PC₁₀₋₀ ← addr11

(3) 相对转移指令

相对转移指令 **SJMP** 也是单操作数指令，功能是把操作数所表示的偏移地址和本指令的下一条指令的地址相加再送入 PC 寄存器，从而实现指令的跳转。短转移指令需要 2 字节，所以最终的目标地址是本指令地址 + 2 + 偏移地址。指令格式如下：

SJMP rel ; PC ← (PC) + 2 + rel

操作数所表示的偏移地址是一个 8 位有符号数，因此它的范围是 -128 ~ 127。SJMP 既可以往前跳（偏移值为正数），也可以往后跳（偏移值为负数）。前跳最远处离本指令距离为 129 字节，后跳最远处离本指令距离为 126 字节。

在无操作系统的软件编程中，为了等待中断或程序结束，常有使程序“原地踏步”的需求，对此可以使用两种格式的 SJMP 指令完成。

LABORG: SJMP LABORG

或

SJMP \$

在汇编语言中，以“\$”代表 PC 的当前值。SJMP \$ 相当于 SJMP -2。

(4) 变址寻址转移指令

变址寻址转移指令 **JMP** 只有一种格式：

JMP @ A + DPTR ; PC ← (A) + (DPTR)

这是一条一字节的转移指令，转移的目的地址由 A 寄存器的内容和 DPTR 内容之和来确定，目的地址 = (A) + (DPTR)。这条指令非常适合于多分支流程。可以将 DPTR 寄存器作为一个基地址，而 A 为变址。A 的内容不同，就可以转移到不同的分支程序中。A 的取值往往和某一个具体的物理特征相关。比如，A 可以表示键盘中各个键的编码值。使用本指令，可以根据所获取键的编码值，转向不同的处理流程。

2. 有条件转移指令

(1) 累加器 A 判零转移指令

累加器 A 判零转移指令 **JZ 或 JNZ** 指令是双字节指令，其功能是首先判断累加寄存器 A 的结果是否为 0，根据判断的结果决定是否转移到目标地址。指令格式如下：

JZ rel	;	若 A 为 0, $PC \leftarrow (PC) + 2 + rel$, 否则, $PC \leftarrow (PC) + 2$
JNZ rel	;	若 A 不为 0, $PC \leftarrow (PC) + 2 + rel$, 否则, $PC \leftarrow (PC) + 2$

【例 3-20】 比较内部 RAM 单元中地址为 60H 的数据如果不为 0，则传送到 P1 口，否则不传送。

```
MOV A, 60H
JZ L1
MOV PL, A
```

L1 :

• • • • •

(2) 比较不等转移指令

比较不等转移指令 **CJNE** 指令是三字节和三操作数指令，其指令形式为：

CJNE <目的操作数>, <源操作数>, rel

其功能是首先将目的操作数和源操作数做一个比较，也就是目的操作数减去源操作数，但结果不回传目的操作数。如果结果不为 0，则转移到目标地址；否则，执行下一条指令。

具体的指令格式如下:

CJNE A, #data, rel	; 若 A = data, PC ← (PC) + 3, 否则, PC ← (PC) + 3 + rel
CJNE A, direct, rel	; 若 A = (direct), PC ← (PC) + 3, 否则, PC ← (PC) + 3 + rel
CJNE Rn, #data, rel	; 若 (Rn) = data, PC ← (PC) + 3, 否则, PC ← (PC) + 3 + rel
CJNE @Ri, #data, rel	; 若 ((Ri)) = data, PC ← (PC) + 3, 否则, PC ← (PC) + 3 + rel

【例 3-21】 内部 RAM 单元中两个存储单元地址为 50H 和 51H，如果它们相等，则将地址为 52H 的数据单元传送给 58H；如果不相等，则将地址为 62H 的数据单元传送给 58H。

```
MOV A, 50H
CJNE A, 51H, L1
MOV 58H, 52H
SJMP L2
```

L1: MOV 58H, 62H

L2 :

• • • • •

(3) 循环减 1 转移指令

循环减 1 转移指令 **DJNZ** 是两字节和两个操作数指令，其指令形式为：

DJNZ <源操作数>, rel

其功能是将源操作数减 1，结果回送到源操作数中去。如果结果不为 0，则转移到 rel 所

例. 把片外 30H 单元开始的数据块传送到片内 RAM 40H 开始的位置
直至出现 0 为止。

```
MOV R0, #30H;
```

```
MOV R1, #40H;
```

```
Loop: MOVX A, @R0;
```

JZ ; <没考虑数据原本就为 0 的情况, 要加 JZ>

```
MOV @R1, A;
```

```
INC R1;
```

```
INC R0;
```

```
JNZ Loop;
```

```
SJMP $.
```

例. 内部 RAM 两单元 50H 和 51H, 若它们相等, 则将地址为 52H 的数据单元传送到 58H;
如果它们不相等, 则将地址为 02H 的数据单元传送到 58H。

```
MOV A, 50H;
```

```
CJNE A, 51H, L1
```

```
MOV 58H, 52H
```

```
SJMP L2
```

```
L1: MOV 58H, 02H
```

```
L2: MOV 58H, 52H
```

例. 统计片内 RAM 30H 单元开始的 20 个数据中 0 的个数, 放于 R7 中.

MOV R0, #30H

MOV R2, #20

MOV R7, #0

Loop: MOV A, @R0

CJNE A, #0, NEXT

INC R7

NEXT: INC R0

DJNZ R2, Loop.

向的地址。

具体的指令格式如下：

DJNZ Rn, rel ; $Rn \leftarrow (Rn) - 1$, 若 $(Rn) = 0$ 则 $PC \leftarrow (PC) + 2$, 否则, $PC \leftarrow (PC) + 2 + rel$

DJNZ direct, rel ; $direct = (direct) - 1$, 若 $(direct) = 0$ 则 $PC \leftarrow (PC) + 2$, 否则, $PC \leftarrow (PC) + 2 + rel$

【例 3-22】 设计一段程序, 在 P1.0 口输出周期为 1ms 的方波。系统晶振频率为 12MHz。因为系统晶振频率为 12MHz, 所以一个机器周期为 $1\mu s$ 。而 DJNZ 是二周期指令, 所以每执行一次, 所需时间为 $2\mu s$ 。方波每半个周期 ($500\mu s$) 取一次反, 而 $500\mu s$ 需要 DJNZ 执行 250 次, 程序如下:

L1: MOV R2, #0FAH

L2: DJNZ R2, L2

CPL P1.0 ; 将 P1.0 引脚取反

SJMP L1

(4) 判 C 转移指令

判 C 转移指令 JC 或 JNC 指令是双字节指令, 其功能是判断 CY 标志位是否为 1, 根据判断的结果决定是否转移到目标地址。指令格式如下:

JC rel ; 若 CY 为 1, 则 $PC \leftarrow (PC) + 2 + rel$, 否则, $PC \leftarrow (PC) + 2$

JNC rel ; 若 CY 不为 1, 则 $PC \leftarrow (PC) + 2 + rel$, 否则, $PC \leftarrow (PC) + 2$

CY 的标志位在无符号数运算中表示进位或借位。特别是在两个无符号数相减后, 如果被减数小于减数, CY 标志为 1, 否则为 0。JC 或 JNC 指令通常和 CJNE 指令相配合, 用于判断两个数的大小关系。

【例 3-23】 比较内部 RAM 单元中两个存储单元的大小。设 A、B、C 的地址分别为 50H、51H、52H。A 和 B 为无符号数, 如果 $A < B$, 则将 -1 送入 C; 如果 $A = B$, 则将 0 送入 C; 如果 $A > B$, 则将 1 送入 C。

MOV A, 50H

CJNE A, 51H, L1

MOV 52H, #0

SJMP L3

L1: JC L2

MOV 52H, #1

SJMP L3

L2: MOV 52H, #-1

L3:

.....

CJNE 指令将 A 和 51H 单元做了一个减法, 影响了 C 标志位, 但是并没有改变任何一个数据的结果。这样为后面的 JC 指令提供了依据。

(5) 判直接寻址位转移指令

判直接寻址位转移指令 JB、JNB 或 JBC 是三字节指令, 其功能是判断一个直接寻址位是否为 1, 根据判断的结果决定是否转移到目标地址。JBC 指令执行后还将该直接寻址位清 0。指令格式如下:

JB bit, rel ; 若 bit 为 1, 则 $PC \leftarrow (PC) + 3 + rel$, 否则, $PC \leftarrow (PC) + 3$

JNB bit, rel ; 若 bit 为 0, 则 $PC \leftarrow (PC) + 3 + rel$, 否则, $PC \leftarrow (PC) + 3$

JBC bit, rel ; 若 bit 为 1, 则 $bit \leftarrow 0$, $PC \leftarrow (PC) + 3 + rel$, 否则, $PC \leftarrow (PC) + 3$

bit 表示一个位寻址, 可以表示内部 RAM 中的位寻址区和专用寄存器的可寻址位。

【例 3-24】 内部 RAM 单元中存储单元地址为 50H, 如果该存储单元有奇数个 1, 则将 R0 置 1; 否则, 将 R0 清 0。编写该程序。

首先将 50H 单元送给 A, 然后判断 P 标志位, 根据 P 标志位的结果, 给 R0 置 1 或清 0。

```
MOV A, 50H
JB P, L1
MOV R0, #0
SJMP L2
L1: MOV R0, #1
L2:
.....
```

3. 子程序调用及返回指令

(1) 子程序绝对调用指令

子程序绝对调用指令 ACALL 是二字节指令, 其功能是首先将 PC 值压入栈中, 然后将操作数所表示的 11 位地址送入 PC 寄存器的低 11 位, 高 5 位不变, 从而实现向子程序的跳转。为了实现子程序的调用, 该指令共完成以下两项操作。

1) 断点保护。断点保护是通过自动的堆栈操作实现的, 即把加 2 后的 PC 值自动送堆栈保存起来, 待子程序返回时再送回 PC。

2) 跳转到目的地址。目的地址的跳转是在 PC 加 2 的基础上, 以指令提供的 11 位地址取代 PC 低 11 位, 而 PC 的高 5 位不变。

指令格式如下:

ACALL addr11

被调用子程序的首地址必须和本指令的下一条指令地址在同一页中 (2KB 一页)。

ACALL 指令的操作内容可表示为:

```
PC ← (PC) + 2
SP ← (SP) + 1, (SP) ← (PC)7-0
SP ← (SP) + 1, (SP) ← (PC)15-8
PC10-0 ← addr11
```

【例 3-25】 有一个子程序的首地址为 LCOUNT, 功能是计算 A 寄存器中 1 的个数, 并将这个计数值放入寄存器 R0。编一程序段, 求出地址为 50H 的 16 位内存单元中 1 的个数, 放入寄存器 A 中。

将这 16 位数据分成 2 字节, 分别传入 A 寄存器中, 然后分别调用子程序计算出它们 1 的个数, 然后再相加到 A 寄存器。

```
MOV A, 50H
ACALL LCOUNT ; 调用子程序
MOV 60H, R0 ; 将计算结果保存在地址为 60H 的内存单元中
MOV A, 51H
ACALL LCOUNT ; 调用子程序
MOV A, R0 ; 将计算结果保存在 A 寄存器中
```

ADD A, 60H ; 将两次调用结果相加

(2) 子程序长调用指令

子程序长调用指令 **LCALL** 是三字节指令, 其功能是首先将 PC 值压入栈中, 然后将操作数所表示的 16 位地址送入 PC 寄存器中, 从而实现向子程序的跳转。指令格式如下:

LCALL addr16

与 ACALL 指令相似, 这条指令包括两个操作, 断点保护和跳转到目标地址。LCALL 指令所调用的子程序的调用范围是 64KB。LCALL 指令的操作内容可表示为:

$PC \leftarrow (PC) + 3$

$SP \leftarrow (SP) + 1, (SP) \leftarrow (PC)_{7-0}$

$SP \leftarrow (SP) + 1, (SP) \leftarrow (PC)_{15-8}$

$PC \leftarrow \text{addr16}$

(3) 子程序返回指令

子程序返回指令 **RET** 和 **RETI** 的功能是跳回到主程序中, 也就是跳回到主程序中子程序调用指令的下一条指令。指令格式如下:

RET ; 子程序返回指令

RETI ; 中断服务子程序返回指令

子程序返回指令的主要操作就是从堆栈中自动取出断点地址送给程序计数寄存器 PC, 使得程序在主程序断点处继续向下执行。RET 指令的操作内容可表示为:

$PC_{15-8} \leftarrow (SP), SP \leftarrow (SP) - 1,$

$PC_{7-0} \leftarrow (SP), SP \leftarrow (SP) - 1,$

中断服务子程序返回指令, 除了上述功能外, 还有清除中断响应时被置位的优先级状态、开放较低级中断和恢复中断逻辑等功能。详细内容请参考第 4 章有关章节。

【例 3-26】 编写一个子程序, 其首地址为 LCOUNT, 功能是计算 A 寄存器中 1 的个数, 并将这个计数值放入寄存器 R0。

程序如下:

LCOUNT: MOV R0, #0; 计数初值为 0

MOV R1, #8 ; R1 是循环变量, 初始值为 8, 因为 A 寄存器有 8 位

LOOPBEGIN: RLC A ; 带进位左移 1 次, 将当前最高位移到进位标志 CY 中

MOV R2, A ; 由于马上要进行加法运算, 保存 A 寄存器的内容到 R2

MOV A, R0 ; 将 R0 送入 A 中, 为下一步加法做准备

ADDC A, #0 ; 将 CY 加入 A 中

MOV R0, A ; 将结果送回 R0 中

MOV A, R2 ; 恢复 A 的值

DJNZ R1, LOOPBEGIN ; 一次循环结束, 将循环计数器减一

RET ; 返回主程序

4. 空操作指令

空操作指令 **NOP** 也算一条控制指令, 即控制 CPU 不做任何操作, 只消耗一个机器周期的时间。空操作指令是单字节指令, 没有任何操作数, 执行后 PC 加 1, 时间延续一个机器周期。NOP 指令常用于程序的等待或时间的延迟。指令格式如下:

NOP ; $PC \leftarrow (PC) + 1$

例. 把片内

3.3.5 位操作指令

和大家所熟悉的 8086 指令系统不同, 51 指令系统本身带有位操作指令。在 8086 指令系统中, 如果更改某一位, 一般原则先读取该位所在字节 (或字), 再用逻辑运算更改, 然后写回, 非常麻烦。而 51 指令系统因为本身带有位操作指令, 可以免去这些麻烦, 这是 51 指令系统的一大特色。

位操作指令中, 所涉及的寻址是位寻址。位操作指令中也有一个累加寄存器, 那就是 CY 标志位。它在位操作指令中的地位相当于普通指令的累加寄存器 A, 有非常重要的作用。

1. 位数据传送指令

位数据传送指令有两种格式:

```
MOV  C, bit      ;CY←(bit)
MOV  bit, C       ;bit←(CY)
```

这里的 bit 是位寻址操作数, 而 C 则表示 CY 标志位。

【例 3-27】 要将 27H 位的内容传送给 26H 位, 不能直接用 “MOV 26H, 27H”, 因为该指令执行的实际是字节传送, 若要将 26H 位的内容传送给 27H 位, 可用下述程序实现:

```
MOV  C, 26H      ;CY←(26H)
MOV  27H, C       ;27H←(CY)
```

2. 位数据清 0 指令

位数据清 0 指令 CLR 可使直接寻址位 bit 或 CY 清 0, 不影响其他标志。指令格式如下:

```
CLR  C           ;CY←0
CLR  bit         ;bit←0
```

【例 3-28】 片内 RAM 单元 26H 的内容为 0FFH, 执行指令:

```
CLR  32H
```

后, 其结果是多少?

解: 26H 单元的值是 0FFH (11111111B), 而 32H 是 26H 单元第二位的位地址。所以, 指令执行后它的值是 11111011B, 26H 的最后的值是 0FBH。

3. 位数据置 1 指令

位数据清 0 指令 SETB 可使直接寻址位 bit 或 CY 置 1, 不影响其他标志。指令格式如下:

```
SETB C          ;CY←1
SETB bit        ;bit←1
```

【例 3-29】 假设进位标志 CY 内容为 0, 输出口 P1 原来的内容为 0FH (00001111B), 则执行下面的指令

```
SETB  C
SETB  P1.7
```

后, P1 口和 CY 的结果是多少?

结果: CY = 1, P1 = 8FH (10001111B)。

4. 位数据求反指令

位数据求反指令 CPL 可使直接寻址位 bit 或 CY 内容取反, 不影响其他标志位。指令格式如下:

```
CPL C           ;CY←/(CY)
CPL bit         ;bit←/(bit)
```

【例 3-30】 执行下面的指令序列：

```
MOV P1, #2FH    ;(P1)←(2FH 即 00101111B)
CPL P1.0        ;P1.0 位求反
CPL P1.2        ;P1.2 位求反
```

执行后，P1 的结果为 2AH (00101010B)。

5. 位与指令

位与指令的功能是将直接寻址位的内容或直接寻址位内容取反后（不改变原来位的内容）和 CY 的内容相与，结果保存在 CY 中。指令格式如下：

```
ANL C, bit      ;CY←(CY) ∧ (bit)
ANL C, /bit     ;bit←(CY) ∧ /(bit)
```

【例 3-31】 当位地址 (2AH) = 1, (32H) = 1, 同时累加器中 (ACC.7) = 0 时，进位标志 CY 置 1；否则，CY 清 0。用程序段实现以上功能。

分析例题可得出，这里需要 3 个条件同时成立，才能保证 CY 为 1，这是典型的逻辑与运算，可使用两条位与指令实现这个关系。

```
MOV C, 2AH
ANL C, 32H
ANL C, /ACC.7
```

6. 位或指令

位或指令的功能是将直接寻址位的内容或直接寻址位内容取反后（不改变原来位的内容）和 CY 的内容相或，结果保存在 CY 中。指令格式如下：

```
ORL C, bit      ;CY←(CY) ∨ (bit)
ORL C, /bit     ;bit←(CY) ∨ /(bit)
```

【例 3-32】 当位地址 (2AH) = 1 或 (32H) = 1 或累加器中 (ACC.7) = 0 时，进位标志 CY 置 1；否则，CY 清 0。用程序段实现以上功能。

分析例题可得出，这里 3 个条件有一个成立，就能保证 CY 为 1，这是典型的逻辑或运算，可使用两条位或指令实现这个关系。

```
MOV C, 2AH
ORL C, 32H
ORL C, /ACC.7
```

3.4 51 系列单片机汇编语言的语句格式

51 系列单片机汇编语言的语句格式如下：

[标号字段:] 操作码字段 [操作数字段] [; 注释字段]

即一条汇编语句由标号、操作码、操作数、注释 4 个部分组成。其中，只有操作码字段是必需的，其他 3 个字段都是可选的。

标号字段是语句所在地址的标志符号。有了标号，程序中的其他语句才能访问该语句。有关标号的规定如下：

- 标号后边必须跟以冒号“:”。
- 由1~8个ASCII字符组成。开头必须是字母，其他字符可以是字符、数字或其他特定字符。
- 同一标号在一个程序中只能定义一次。
- 不能使用汇编语言已经定义的符号作为标号。

操作码字段是汇编语言指令中唯一不能空缺的部分，它规定了语句的操作内容。操作码是以指令助记符或伪指令助记符表示的。

操作数字段用于给指令的操作码提供数据或地址。对于指令助记符表示的操作码，通常有单操作数、双操作数和无操作数3种情况。但是对于伪指令助记符表示的操作码，操作数的个数可能有多。如果有多个操作数，则操作数之间，要以逗号隔开。

注释字段不属于语句的功能部分，它只是对语句的解释说明，是为了方便程序的阅读而使用的。以分号“;”开头，表明本行的后续部分为注释内容。注释的长度不够，可以换行，但新的行也应“;”开头。

3.5 51 系列单片机汇编程序常用伪指令

有一些指令，如指定目标程序或数据存放的地址、给一些指定的标号赋值、表示源程序结束等指令，并不产生目标程序（机器码），也不影响程序的执行，仅仅产生供汇编用的某些命令，用来对汇编过程进行某种控制或操作，这类指令称为伪指令。如果是手工编译，这些伪指令将失去意义。

3.5.1 定义起始地址伪指令

定义起始地址伪指令ORG的功能是规定一个程序块或数据块所存放的起始地址，也就是紧随其后的程序段或数据块将被汇编工具安置在所规定的起始地址。指定格式如下：

ORG addr16

addr16表示一个16位的程序存储器的空间地址，一般为一个确定的地址，也可以是事先定义的标号。

【例 3-33】

ORG 2000H

START: MOV A, 30H

当汇编工具汇编完这段程序后，会把“MOV A, 30H”这条指令安置在程序存储器中地址为2000H的存储空间中。START这个标号所代表的地址将为2000H。

3.5.2 定义汇编结束伪指令

定义汇编结束伪指令END的功能是表示汇编程序到此结束。在整个源程序中只能有一条END命令，且位于程序的最后。指令格式如下：

END

3.5.3 标号赋值伪指令

标号赋值伪指令EQU的功能是将表达式的值赋给本语句中的标号，又称为等值指令。

指令格式如下：

标号 EQU [表达式]

表达式的值为 8 位或 16 位二进制数，赋值以后的字符名称即可以作为地址使用，也可以作为立即数使用。

【例 3-34】

TEST EQU 2000H

表示标号 TEST = 2000H，在汇编时，凡是遇到标号 TEST 时，均以 2000H 来代替。

3.5.4 字节定义伪指令

字节定义伪指令 DB 的功能是在程序存储器的连续单元中定义字节数据。指令格式如下：

标号: DB [字节表]

标号表示所定义字节数据的起始地址，标号这一项也可没有。字节表由若干字节常数、字符、字符串构成，字节表中的两个连续元素用逗号分开。单个字符或字符串中的每个字符在程序存储器中由对应的 ASCII 码表示。

【例 3-35】

ORG 30H

DB -2,1,45,'A'

当汇编结束后，程序存储器的 30H 存储单元的内容为 FEH（-2）；程序存储器的 31H 存储单元的内容为 01H；程序存储器的 32H 存储单元的内容为 45；程序存储器的 33H 存储单元的内容为 41H（'A'）。

3.5.5 字定义伪指令

字定义伪指令 DW 的功能是在程序存储器的连续单元中定义 16 位的字数据。指令格式如下：

标号: DW [字表]

字表由若干字常数、字符、双字符字符串构成，字表中的两个连续元素用逗号分开。每个字数据在存储器中是高位字节在前，低位字节在后。单个字符在存储器中按顺序存放 00H 和该字符对应的 ASCII 码；而双字符字符串则按顺序将两个字符对应的 ASCII 码存放在存储器中。

【例 3-36】

ORG 60H

DW 23,'AB',4566H,'5'

当汇编结束后，程序存储器的 60H 存储单元的内容为 00；程序存储器的 61H 存储单元的内容为 17H（23）；程序存储器的 62H 存储单元的内容为 41H（'A'）；程序存储器的 63H 存储单元的内容为 42H（'B'）；程序存储器的 64H 存储单元的内容为 45H；程序存储器的 65H 存储单元的内容为 66H；程序存储器的 66H 存储单元的内容为 00H（'c'）；程序存储器的 67H 存储单元的内容为 35H（'5'）。

3.5.6 存储区定义伪指令

存储区定义伪指令 DS 的功能是在程序存储器中定义一个存储区。指令格式如下：

标号: DS [表达式]

标号表示所定义存储区的起始地址, 标号这一项也可没有。表达式的值表示这个存储区要预留多少个存储单元。

【例 3-37】

```
ORG 2100H
```

```
DS 08H
```

当汇编结束后, 从 2100H 地址开始保留 8 个地址连续的字节。

3.5.7 位定义伪指令

位定义伪指令 BIT 的功能是将位地址赋予一个字符名称。指令格式如下：

字符名称 BIT [位地址]

位地址可以是绝对地址, 也可以是已经定义的位地址符号。

【例 3-38】

```
AQ BIT P1.0
```

```
BQ BIT AQ
```

把 P1.0 的位地址赋予 AQ, 然后又把 AQ 赋予 BQ。在其后的编程中, AQ 和 BQ 都可以作为 P1.0 这个位地址使用。

3.5.8 内部 RAM 地址赋值伪指令

内部 RAM 地址赋值伪指令 DATA 用于将一个内部 RAM 地址赋给指定的符号名。指令格式如下：

符号名 DATA 表达式

数值表达式的值应该在 0 ~ 255 之间, 表达式应该是一个简单再定位表达式。

【例 3-39】

```
REGBUF DATA 40H ;将 REGBUF 指定为内部 RAM 地址 40H
```

```
MOV 40H, #34H
```

```
MOV A, REGBUF
```

执行以上程序段后, A 的值是 34H。

3.5.9 外部 RAM 地址赋值伪指令

外部 RAM 地址赋值伪指令 XDATA 把片外数据地址赋予规定的符号名称。表达式应该是一个简单再定位表达式。如果该表达式后的求值不是一个数, 它的段类型一定是 XDATA。其符号取段类型 XDATA。由 XDATA 定义的符号不得在程序中别的地方重新定义。指令格式如下：

符号名 XDATA 表达式

【例 3-40】

```
REGBUF XDATA 4540H ;将 REGBUF 指定为外部 RAM 地址 4540H
```

```

MOV  DPTR, #4540H
MOV  A, #25H
MOVX @DPTR, A
MOV  DPTR, #REGBUF
MOVX A, @DPTR

```

执行以上程序段后, A 的值为 25H。

3.6 51 系列单片机汇编程序设计方法

和大家熟悉的高级语言设计一样, 汇编程序的设计也常采用以下几种基本结构: 顺序结构、分支结构和循环结构, 再加上广泛使用的子程序和中断服务子程序。任何复杂的程序都是由这几种基本结构复合而成的。

一个高质量的程序应具有以下特点。

- 程序有较好的逻辑结构, 便于二次开发。
- 源程序有较好的可读性, 使非专业人员能读懂会用, 甚至能加以修改。
- 程序结构应有较好的可靠性和可维护性。也就是说, 要保证程序能正确地工作, 并且易于进一步的改进和完善。
- 程序运行效率高而且可重用。

3.6.1 顺序结构程序设计

顺序结构是最简单的一种基本结构, 是指完全按顺序逐条执行的指令序列。

【例 3-41】 已知 X、Y、Z 分别为片内 RAM 30H、31H、32H 单元的内容, 设 $X > Y$, 试编程完成下式的算术运算: $S = (X - Y) Z$, 并将计算结果 S 存入片内 RAM 34H (高字节)、35H (低字节) 单元中。

由于 $X > Y$, 所以 $X - Y > 0$, 不需要借位; $(X - Y) Z$ 的结果最多占用 2 字节。首先将 $X - Y$ 结果放入 A 寄存器中, 将 Z 的内容放入 B 寄存器中, 然后将 A 和 B 相乘, 其结果的高低字节分别置于 B 和 A 寄存器中, 然后将它们分别放入 34H 和 35H 存储单元中。

```

ORG 0030H
START: MOV  A, 30H      ;A←X
        CLR  C
        SUBB A, 31H      ;A←(A) - X
        MOV  B, 32H      ;B←Z
        MUL  AB
        MOV  34H, B      ;34H←A * B 的高字节
        MOV  35H, A      ;35H←A * B 的低字节
        SJMP $

```

3.6.2 分支结构程序设计

在 51 指令系统的编程中, 单分支程序使用条件转移指令来实现, 即根据条件对程序的执行进行判断, 满足条件就进行程序跳转, 否则按顺序执行。

【例 3-42】 假设外部 RAM 中有 3 个 8 位无符号数据，它们的地址分别为 7111H、5687H 和 8766H，找出它们之中最大的数放入寄存器 A 中。

先将地址为 7111H、5687H 的两个数相比较，将其中较大的数放置于 R0；然后将 R0 和地址为 8766H 的数相比较，将其中较大数置于 A 中。

```
ORG 0000H
MOV DPTR, #7111H
MOVBX A, @DPTR
MOV R0, A
MOV DPTR, #5687H
MOVBX A, @DPTR
MOV R1, A
CLR C
SUBB A, R0
JC BIG1
MOV A, R1
MOV R0, A
BIG1: MOV DPTR, #8766H
MOVBX A, @DPTR
MOV R2, A
CLR C
SUBB A, R0
JC BIG2
MOV A, R2
SJMP LABLAST
BIG2: MOV A, R0
LABLAST: SJMP $
END
```

3.6.3 循环结构程序设计

循环是为了重复执行一个程序段。在 51 指令系统的汇编语言中，循环次数固定的循环常常采用 DJNZ 指令来实现。在循环初始化时，将循环次数放置于某个通用寄存器中或某个内部 RAM 单元中；在循环结束处放置一条 DJNZ 指令，可同时完成对循环计数单元的减 1 和条件跳转功能（如果循环计数单元不为 0，则跳转到循环程序段的起始处）。对于循环次数依赖于循环体中执行结果的循环程序，则需要其他类型的有条件跳转指令。

【例 3-43】 外部 RAM 中，从地址 2000H 开始存放了 100 个有符号字节数据，编写一段程序，统计其中小于 0 的数据个数，将其存放在 R0 寄存器中。

显然，该程序是一个循环次数固定的程序，循环次数是 100 次。将 R1 作为循环计数寄存器，其初始值为 100。在循环体中是一个分支结构，用于判定一个数是否小于 0 的依据是该数的最高位是否为 1。

程序如下：

```
ORG 0000H
```

```

MOV R0, #0
MOV R1, #100
MOV DPTR, #2000H
LOOP: MOVX A, @DPTR
      INC DPTR
      JNB ACC.7, NEXT      ;如果 A 最高位为 0,则表明大于或等于 0,不计入
      INC R0
NEXT: DJNZ R1, LOOP        ;将 R1 减 1,如果大于 0,表明循环继续
      SJMP $
      END

```

【例 3-44】 一个字符串存放在外部 RAM 中,起始地址为 3000H,求这个字符串的长度,放入 R0 寄存器中。

根据字符串的原理,将字符串的每个字符所对应的 ASCII 码按顺序存放在存储器中,字符串的结尾是一个 ASCII 码为 0 的结尾符。显然,该程序是一个循环次数不固定的程序,该循环的结束条件是从外部 RAM 中得到的当前字符的 ASCII 码是否为 0。如果是 0,表明该字符串到此结束。

程序如下:

```

ORG 0000H
MOV R0, #-1      ;在循环中,至少要进行一次加法,所以计数值初始值为 -1
MOV DPTR, #3000H
LOOP: MOVX A, @DPTR      ;将当前字符的 ASCII 码送入 A 寄存器
      INC DPTR
      INC R0
      CJNE A, #0, LOOP    ;如果当前字符的 ASCII 码不为 0,循环继续
LEND: SJMP $
      END

```

3.6.4 查表程序设计

在程序设计中,查表功能是一项非常重要的功能。查表就是根据自变量 x ,在表格中寻找 y ,使 $y=f(x)$ 。自变量 x 的范围一般是一个连续变化的整数,比如 0~99 之间的整数。因此,将所对应的 y 值按自变量 x 的顺序排列在一起,就得到了一个线性表。根据 x 的值求取 y 值,只需要在这个线性表中查询相应的单元即可,称之为查表功能。

在 MCS-51 的指令系统中,给用户提供了两条极为有用的查表指令:

```

MOVC A, @A + DPTR
MOVC A, @A + PC

```

从这两条指令的特性可以看出,被查询的线性表只能建立在程序存储空间。

指令“MOVC A, @A + PC”以 PC 作为基址寄存器,PC 的内容和 A 的内容作为无符号数,相加后所得的数作为某一程序存储器单元的地址,根据地址取出程序存储器相应单元中的内容送到累加器 A 中。

指令“MOVC A, @A + DPTR”完成把 A 中的内容作为一个无符号数与 DPTR 中的内容相加,所得结果为某一程序存储单元的地址,然后把该地址单元中的内容送到累加器 A 中。

“MOVC A, @ A + DPTR”这条指令的应用范围较为广泛,一般情况下,大多使用该指令。使用该指令时不必计算偏移量,使用该指令的优点是表格可以设在 64KB 程序存储器空间内的任何地方,而不像“MOVC A, @ A + PC”那样只设在 PC 下面的 256 个单元中,使用较方便。

【例 3-45】 在一个以 MCS-51 为核心的温度控制器中,温度传感器输出的电压与温度为非线性关系,传感器输出的电压已由 A/D 转换为 8 位二进制数。根据测得的不同温度下的电压值数据构成一个表,表中放温度值 y 为双字节无符号数,单字节无符号数 x 为电压值数据。设测得的电压值 x 放入 R2 中,编写一段程序,根据电压值 x ,查找对应的温度值 y ,放入 R2R3 中。

这是一个典型的查表程序。首先使用 DW 伪指令按电压值的顺序建立一个电压温度表。因为温度值是双字节数,因此这个表的每一项占 2 字节,高位字节在前,低位字节在后。表的基地址存放在 DPTR 寄存器中。查表时,首先将表的索引电压值乘以 2,再进行查询。电压值乘以 2 后,有可能大于 255,产生 CY 进位。如果产生了 CY 进位,则应通过将 DPH 加 1 的方法进位处理后再查询。

```

ORG 0000H
MOV DPTR, #TAB1    ;将表的基地址存放在 DPTR 寄存器中
MOV A, R2           ;将电压值放入 A 寄存器中
CLR C              ;清除 CY 标志位
RLC A              ;将 A 乘以 2
JNC LNOCY          ;如果没有进位则跳过进位处理语句
INC DPH            ;将 DPH 加 1,也就是将 DPTR 加 256,处理了进位
LNOCY: MOV R4, A    ;保存索引值 A 到 R4 寄存器
MOVC A, @ A + DPTR ;查表得到温度值的高字节
MOV R2, A          ;将温度值的高字节置于 R2 寄存器中
MOV A, R4          ;恢复索引值 A
INC A              ;索引值 A 加 1
MOVC A, @ A + DPTR ;查表得到温度值的低字节
MOVC R3, A         ;将温度值的低字节置于 R3 寄存器中
LEND: SJMP LEND
TAB1: DW .....    ;温度值表

```

还可以使用查表的方式设计多分支程序,所使用的指令是“JMP @ A + DPTR”。其中 DPTR 寄存器存放的是分支地址表的首地址, A 寄存器存放的是分支地址表的索引。

使用查表的方式设计多分支程序的原理是:假设一个程序中有 n 个分支,每一个分支的入口地址分别是 $BR0, BR2, \dots, BR_{n-1}$,建立一个分支表,该表的首地址置于 DPTR 寄存器,共有 n 项。任意第 i ($0 \leq i \leq n-1$) 项的内容只有一条语句, AJMP BR i , 占 2 字节。将 $2i$ 的值置于 A 寄存器,则执行指令“JMP @ A + DPTR”后,下一条指令将跳转到分支表的第 i 项。而分支表第 i 项所载有的指令是“AJMP BR i ”,所以程序最后将跳转到第 i 个分支。因此,利用分支程序表,只需要执行一次跳转语句“JMP @ A + DPTR”就可以跳转到任何分支。

【例 3-46】 以下程序利用查表的方式设计 4 分支程序。A 的内容为 0~4。

```

        ORG 0000H
        MOV DPTR, #BRTAB
        RLA
        JMP @ A + DPTR
LEND:   SJMP $
BRTAB:  AJMP BR0
        AJMP BR1
        AJMP BR2
        AJMP BR3
BR0:    ...
        SJMP LEND
BR1:    ...
        SJMP LEND
BR2:    ...
        SJMP LEND
BR3:    ...
        SJMP LEND
        END

```

3.6.5 子程序设计

子程序又称为过程，它相当于高级语言中的过程和函数。在一个程序的不同部分往往要用到类似的程序段，这些程序段的功能和结构形式相同，只是某些变量的赋值不同，此时可以把这些程序段写成子程序的形式，以便需要时可以调用它。一般称调用子程序的那段程序为主程序。

子程序的第一条指令的地址称为子程序的入口地址，该指令前必须有标号。子程序的最后一条指令必须是 RET 指令。子程序可以嵌套，即子程序可以调用子程序。

在 51 指令系统中，调用子程序的指令是 ACALL 和 LCALL。子程序的调用需要注意以下几点。

1. 保存与恢复寄存器

由于主程序和子程序经常是分别编写的，所以它们所使用的寄存器会发生冲突。主程序在调用子程序以前，要保护那些可能被子程序破坏的寄存器。在结束子程序运行前，要恢复那些被保护的寄存器。一般采用压栈和出栈的方法实现寄存器的保护和恢复。

2. 子程序的参数传送

主程序在调用子程序时，经常需要传送一些参数给子程序，成为输入参数；子程序运行完后经常要回送一些信息给主程序，称为输出参数。这种主程序和子程序之间的信息传送方式称为参数传送。参数传送常用的方法包括寄存器传送方式、地址表传送方式和堆栈传送方式。

【例 3-47】 已知寄存器 R0、R1、R2 存放的是 8 位二进制数，分别将它们转换成 BCD 码，存放在内部 RAM 的 60H ~ 65H 单元。8 位二进制数的对应 BCD 码是 3 位（百位、十位、个位）十进制数，总共 12 位（12bit），所以用 2 字节来表示对应的 BCD 码。十位和个位分

别存放在第一个字节的高4位和低4位，百位存放在第二个字节的低4位。第二个字节的高4位为全0。

编写一个子程序，该子程序的功能是转换一个8位二进制数为BCD码。该子程序的输入参数是：待转换的单字节数在累加器A中。输出参数是：转换后的BCD码整数（十位和个位）仍在累加器A中，百位在R3中。根据题意，将该子程序调用3次，每次的结果放置到相应的内存单元中。

程序如下：

```
ORG    0000H
MOV     A, R0
ACALL  CHBCD
MOV     60H, A      ; 将十位数和个位数置于存储器中
MOV     61H, R3     ; 将百位数置于存储器中
MOV     A, R1
ACALL  CHBCD
MOV     62H, A      ; 将十位数和个位数置于存储器中
MOV     63H, R3     ; 将百位数置于存储器中
MOV     A, R2
ACALL  CHBCD
MOV     64H, A      ; 将十位数和个位数置于存储器中
MOV     65H, R3     ; 将百位数置于存储器中
LEND:   SJMP    $
CHBCD:  PUSH PSW    ; 保护现场
        PUSH B
        MOV B, #100
        DIV AB      ; A←BCD 码的百位
        MOV R3, A    ; R3←BCD 码的百位
        MOV A, #10   ; 余数继续分离十位和个位
        XCH A, B
        DIV AB      ; A←BCD 码的十位
        SWAP A
        ORL A, B     ; 将十位和个位拼装成BCD码
        POP B        ; 恢复现场
        POP PSW
        RET
END
```

习 题 三

1. MCS-51 单片机有哪几种寻址方式？各寻址方式所对应的寄存器和存储空间有何不同？
2. MCS-51 的指令系统按功能可分为哪几类？
3. 若 (50H) = 60H，写出执行下面程序段后累加器 A、寄存器 R0，以及内部 RAM 的 60H、61H、62H 单元中的内容。
MOV A, 50H

```
MOV R0, A
MOV A, #00H
MOV @R0, A
MOV A, 3BH
MOV 61H, #60H
MOV 62H, A
```

4. 一个 16 位数的高字节存放在 30H 单元, 低字节存放在 31H 单元, 另一个 16 位数据的高字节存放在 32H 单元, 低字节存放在 33H 单元, 试编写程序完成这两个 16 位数据的减法运算, 差的高 8 位存入 40H, 低 8 位存放到 41H 单元。

5. 编写一段程序, 查找存放在 30H ~ 50H 单元中是否有数据 34H, 若有则将 F0 置 1, 否则, 将 F0 清 0。

6. 编写程序, 求出片内 40H 单元内容中所有“0”的个数, 结果存入 50H。

7. 在内部 RAM 单元从 30H 开始的 5 个单元中, 存放着 5 个压缩 BCD 码, 编写一段程序, 将它们转换成 ASCII 码, 并存放到 40H 开始的单元中。

8. 编程求出内部 RAM 中从 30H 开始的连续 20 个单元中的内容的平均值, 并存入 60H 单元中。

9. 编程完成以下功能: 检测内部 RAM 单元从 30H 开始的 32 个数, 若为正数或 0, 则值不变; 若为负数, 则取补后送回。

10. 在内部 RAM 单元中, 从 30H 开始连续存放着 20 个 8 位无符号数, 编程排列其顺序, 使它们从 20H 单元开始, 按照从大到小的顺序依次存放。